

Regfile 寄存器/命令文件格式规范

`.Regfile` 是基于 JSON 语法的配置文件，用于向 I2C 和 SPI 协议分析器（Oscilloscope 等应用）提供目标设备的寄存器/命令定义、说明、访问权限、协议相关参数（如线宽、Dummy 等）以及字段解码映射。

挂载 `.Regfile` 文件后，分析器可直接在图形界面中对截获的读写报文（例如 I2C 的寄存器读写，或 SPI 的各类 OpCode 指令解析）进行位（Bit）级别的解析和释义翻译，免去了工程师一边查波形一边对照芯片数据手册的繁琐工作。

1. 根级别结构 (Root Structure)

每个 `.Regfile` 文件的顶层必须是一个合法的 JSON 对象，包含以下基本字段：

```
{
  "name": "SN65DP159",
  "addresses": ["0x5E", "0x5F"],
  "hasSubaddress": false,
  "addressMap": {
    "0x5C": "A1=1, A0=0",
    "0x5B": "A1=1, A0=1"
  },
  "registers": {
    ...
  }
}
```

字段名	类型	说明
<code>name</code>	String (必填)	设备名称或型号名称，用于在 UI 的挂载列表和标题栏中展示。
<code>addresses</code>	Array[String/Int] (选填)	（主要用于 I2C）该芯片支持的地址列表（十六进制字符串或整数）。当总线上出现匹配的地址时，系统可能会自动将此文件作为匹配候选项。SPI 设备可忽略。

字段名	类型	说明
<code>addressMap</code>	Object (选填)	(主要用于 I2C) 定义地址到 外围硬件配置引脚 的映射。在波形解析时不仅能匹配设备地址，还能在 UI 标题上提示当前的硬件接线方式 (如 <code>[Pin Config: A1=1, A0=0]</code>)。它与 <code>addresses</code> 选其一即可。
<code>hasSubaddress</code>	Boolean (选填)	芯片是否使用寄存器子地址 (默认为 <code>true</code>)。部分 I2C 芯片 (如 Switch/IO 扩展芯片, 如 TCA9548A) 写操作没有子地址阶段, 第一个 Byte 即为数据。若设为 <code>false</code> , 界面将会把所有数据映射到 <code>0x00</code> 地址。对于部分无需子地址的 SPI 设备也可设为 <code>false</code> 。
<code>registers</code>	Object (必填)	寄存器/指令映射字典。其键 (Key) 为寄存器的子地址 (I2C) 或命令 OpCode (SPI), 其值 (Value) 为对应的详细定义。

2. 寄存器/命令定义 (Register / Command Definition)

`registers` 字典中的每一个 Key 是一个表示十六进制的字符串。对于 **I2C 设备**, 这通常代表寄存器的子地址 (Sub-address); 对于 **SPI 设备**, 这通常代表指令操作码 (OpCode, 如 `"0x06"` 代表 Write Enable, `"0xEB"` 代表 Fast Read Quad I/O)。Value 为该寄存器/命令的属性对象。

```
// I2C 寄存器示例
"0x20": {
  "name": "AUX Output Control",
  "access": "R/W",
  "description": "AUX Output Control Register.",
  "fields": [
    ...
  ]
}
```

```
// SPI 命令示例 (W25Q64JW)
"0xEB": {
  "name": "Fast Read Quad I/O",
  "access": "R",
  "addrLines": 4,
  "modeClocks": 2,
  "dummyClocks": 4,
```

```

    "dataLines": 4,
    "addrBytes": 3
  }

```

字段名	类型	说明
<code>name</code>	String (必填)	寄存器/命令的名称（如 <code>DEVICE_ID</code> 或 <code>Fast Read</code> ）。
<code>access</code>	String (选填)	整体访问权限。常用的如 <code>R</code> (读), <code>W</code> (写), <code>R/W</code> (读写)。对于 SPI OpCode, 该属性也可表示数据阶段总线的流向。
<code>description</code>	String (选填)	寄存器/命令的功能说明。将在解析窗口中长文本展示, 可以直接从 Datasheet 摘抄。
<code>fields</code>	Array[Object] (选填)	包含该寄存器所有位字段 (Bit-field) 定义的数组（常用于 I2C 寄存器或 SPI 的状态寄存器）。
<code>dummyClocks</code>	Int (选填, SPI特有)	该指令所需的 Dummy 时钟周期数。
<code>addrBytes</code>	Int (选填, SPI特有)	该指令所需的地址字节数（通常为 3 或 4）。
<code>addrLines</code>	Int (选填, SPI特有)	地址阶段使用的 SPI 数据线数量（如 1=Standard, 2=Dual, 4=Quad）。
<code>dataLines</code>	Int (选填, SPI特有)	数据阶段使用的 SPI 数据线数量（如 1, 2, 4）。
<code>modeClocks</code>	Int (选填, SPI特有)	Mode / Continuous Read 模式的时钟周期数。

3. 位字段定义 (Field Definition)

`fields` 数组规定了此寄存器在一个 Byte (8 bit) 中不同位段的具体含义。支持为每个字段单独编写映射表。

```

{
  "name": "AUX_SWING",
  "startBit": 0,
  "endBit": 2,
  "access": "RW",

```

```
    "description": "Swing Control for AUX Output",
    "valueMap": {
      "0": "270mV",
      "1": "355mV",
      "2": "450mV",
      "7": "Not allowed"
    }
  }
}
```

字段名	类型	说明
name	String (必填)	位段的缩写或名字（如 <code>AUX_SWING</code> ）。
startBit	Int (必填)	位段的起始位（0~7）。如果是单比特字段，则 <code>startBit</code> 和 <code>endBit</code> 相同。
endBit	Int (必填)	位段的结束位（0~7）。要求 <code>endBit >= startBit</code> 。
access	String (选填)	该位段的单独权限（如只读、读写、读后清零等）。
description	String (选填)	该位段的详细解释。
valueMap	Object (选填)	字段值映射表 。键（Key）必须是十进制数字的字符串，值（Value）是对应的释义。例如读取该位段的结果为 <code>0</code> 时，将在界面上翻译成 <code>"270mV"</code> 。如果没有 <code>valueMap</code> ，界面将只显示提取的二进制数据。

4. 界面解析效果

编写良好的 `.Regfile` 可以在解析时呈现如下 Verilog 风格及释义排版：

Device: SN65DP159 (0x5E) [Pin Config: A1=0, A0=0] - Write Reg 0x20 AUX Output Control
Access: R/W AUX Output Control Register.

`0x05` (写入的原始值)

- `Bit[3]` `1'b0` `AUX_TX_SR` (AUX_TX_SR Slew Rate Control...)
- `Bit[2:0]` `3'b101` `710mV` (AUX_SWING; Swing Control...)

5. 编写建议

1. **保留地址 (Reserved)**：某些未指定的地址可能会被用作伪操作或页切换寄存器。如果 Datasheet 没有定义某个地址，建议将它们统一定义为 `Reserved` 或直接标明类似 `"Page Index Select"` 这样在 TI 芯片中常见的特定操作，防止在总线抓包时被识别为 `"Unknown"` 报错。
2. **Key 格式规范**：虽然解析器内部有容错处理，但是推荐所有的十六进制寄存器地址 (Key) 和映射表键都加上双引号并保证字符串规范，如 `"0x00"` 或 `"0x0C"`。